CAPSTONE PROJECT REPORT

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B. TECH CSE (CORE)

Database & Management System Lab (ENCS254)

Submitted by-

Priyanshi Patel

Roll no - 2401010268

Submitted to-

Mansi Kajal

Music Library Management System

1. Introduction

The **Music Library Web Application** is a full-stack web-based system designed to provide users with a seamless and interactive platform for managing and streaming music. In today's digital era, music consumption has shifted significantly from traditional storage methods to online streaming platforms. This project aims to replicate the core functionalities of modern music streaming applications while focusing on database design, system integration, and user experience.

The application allows users to explore a collection of songs categorized by artists, albums, and genres. It provides features such as music playback, search functionality, playlist creation, and favorites management. Users can register and log in to access personalized features, making the system more dynamic and user-centric. The goal is to build a structured and scalable system that demonstrates how real-world music platforms operate at a fundamental level.

From a technical perspective, the project follows a **three-tier architecture**, consisting of the frontend, backend, and database layers. The frontend is developed using HTML, CSS, and JavaScript to create a responsive and visually appealing user interface. The backend is implemented using Node.js and Express, which handle API requests, business logic, and communication between the client and the database. The database is built using MySQL, ensuring efficient data storage and retrieval through well-defined relational schemas and foreign key constraints.

One of the key objectives of this project is to demonstrate the practical application of **Database Management System (DBMS)** concepts such as normalization, relationships, indexing, and query optimization. The system maintains multiple related tables such as users, artists, albums, tracks, playlists, and favorites, ensuring data integrity and consistency across the application.

In addition to technical implementation, the project also emphasizes user experience by incorporating a modern UI design inspired by popular platforms. Features like a dynamic music player, hover-based play controls, and an organized layout contribute to an engaging and intuitive interface.

Overall, this Music Library Web Application serves as a comprehensive demonstration of full-stack development, combining database design, backend logic, and frontend interactivity into a single integrated system. It not only fulfills academic requirements but also provides a strong foundation for building more advanced and scalable music streaming platforms in the future.

2. System Architecture

The **Music Library Web Application** follows a **three-tier architecture**, which separates the system into three independent layers: **Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database)**. This architecture improves scalability, maintainability, and performance by clearly defining responsibilities for each layer.

2.1 Overview of Architecture

The overall flow of the system is as follows:

User → Frontend (UI) → Backend (API Server) → Database → Backend → Frontend → User

This flow ensures that all user interactions are processed securely and efficiently.

2.2 Presentation Layer (Frontend)

The **frontend layer** is responsible for interacting with the user and displaying the data in a structured and visually appealing format.

Technologies Used

- HTML (structure of web pages)
- CSS (styling and layout)
- JavaScript (dynamic behavior and API calls)

Key Responsibilities

- Display music tracks, artists, albums, and playlists
- Provide search functionality
- Handle user interactions (click, play, add to playlist)
- Show music player controls (play, pause, next, previous)
- Send API requests to the backend using Fetch API

Important Components

- **Landing Page** → Entry point of application
- **Login/Register Page** → User authentication
- **Home Page** → Displays all tracks
- **Artist Page** → Shows songs by specific artist
- **Album Page** → Shows songs from selected album
- **Playlist Page** → User-created playlists
- **Music Player Bar** → Controls playback

2.3 Application Layer (Backend)

The **backend layer** acts as the brain of the system. It processes user requests, applies business logic, and communicates with the database.

Technologies Used

- Node.js (runtime environment)
- Express.js (web framework)

Key Responsibilities

- Handle HTTP requests (GET, POST, PUT, DELETE)
- Authenticate users (login/register)
- Process search queries
- Manage playlists and favorites
- Fetch and send data to frontend in JSON format

Main API Endpoints

- /tracks → Fetch all songs
- /artists → Manage artist data
- /albums → Fetch album data
- /playlists → Create and manage playlists
- /login → User authentication
- /register → User registration
- /search → Search songs

2.4 Data Layer (Database)

The **database layer** stores all application data in a structured format using relational tables.

Technology Used

- MySQL

Key Responsibilities

- Store users, songs, artists, albums, playlists
- Maintain relationships using foreign keys
- Ensure data consistency and integrity

Main Tables

- **Users** → Stores user credentials

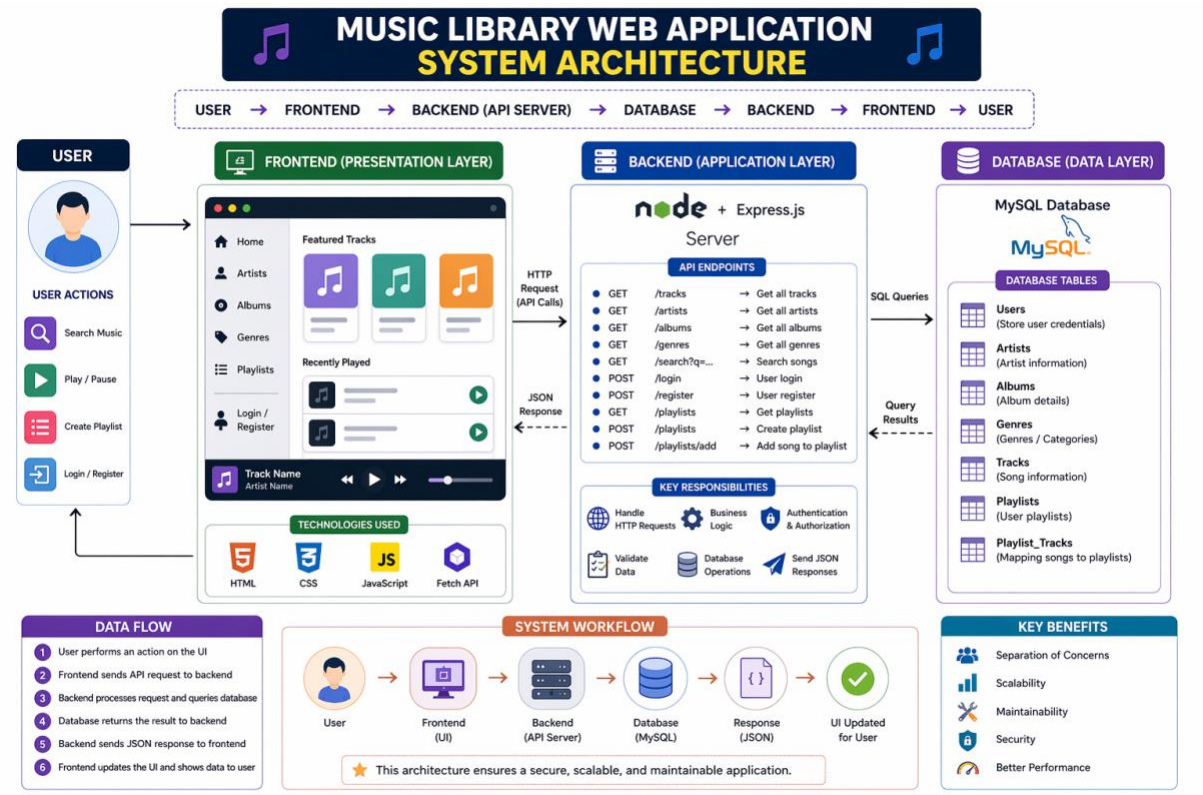
- **Artists** → Artist details
- **Albums** → Album information
- **Tracks** → Song details
- **Genres** → Song categories
- **Playlists** → User playlists
- **Playlist_Tracks** → Mapping songs to playlists

2.5 Data Flow Explanation

1. User performs an action (e.g., search song)
2. Frontend sends request to backend API
3. Backend processes request and queries database
4. Database returns result to backend
5. Backend sends response (JSON) to frontend
6. Frontend updates UI dynamically

2.6 Architecture Diagram

▼



2.7 Advantages of This Architecture

- **Separation of Concerns** → Each layer works independently
- **Scalability** → Easy to upgrade backend or frontend separately
- **Maintainability** → Code becomes organized and manageable
- **Security** → Database is not directly exposed to users
- **Performance** → Efficient data handling using APIs

3. OBJECTIVE

The main objectives of this project are:

- To design a relational database for a music system
- To implement CRUD operations using REST APIs
- To build a responsive frontend interface
- To integrate audio playback functionality
- To allow users to manage playlists and favorites

4. DATABASE DESIGN

The database consists of the following tables:

1. Artists

- artist_id (Primary Key)
- name
- bio
- image_url

2. Albums

- album_id (Primary Key)
- title
- artist_id (Foreign Key)
- release_year
- image_url

3. Genres

- genre_id (Primary Key)
- name

4. Tracks

- track_id (Primary Key)
- title
- album_id (Foreign Key)
- genre_id (Foreign Key)
- duration
- audio_url
- image_url

5. Playlists

- playlist_id (Primary Key)
- name
- created_at

6. Playlist_Tracks

- playlist_id (Foreign Key)
- track_id (Foreign Key)

7. Users

- user_id (Primary Key)
- username
- password

8. Favorites

- user_id
- track_id

5. BACKEND FUNCTIONALITY

The backend is developed using Node.js and Express.js.

Key APIs:

Artists:

- GET /artists
- POST /artists
- PUT /artists/:id
- DELETE /artists/:id

Tracks:

- GET /tracks

- POST /tracks
- PUT /tracks/:id
- DELETE /tracks/:id

Albums:

- GET /albums

Genres:

- GET /genres

Playlists:

- GET /playlists
- POST /playlists
- POST /playlists/add

Authentication:

- POST /register
- POST /login

Search:

- GET /search?q=

6. SEARCH FEATURES

The application supports:

- Search by song title
- Search by artist
- Search by genre

These are implemented using SQL queries with JOIN operations.

7. ANALYTICAL QUERIES

The following insights are generated:

- Most popular genre
- Number of songs per artist
- Playlist song count

8. FRONTEND DESIGN

The frontend includes:

- Landing page
- Login/Register page
- Home page (tracks display)
- Artists page
- Albums page
- Playlist page

Features:

- Search bar
- Music cards
- Audio player
- Playlist dropdown
- Add to favorites

9. USER INTERFACE

The UI is designed with:

- Sidebar navigation
- Card-based layout
- Glassmorphism effects
- Hover animations
- Bottom music player

10. AUTHENTICATION SYSTEM

Users can:

- Register an account
- Login using credentials

Basic authentication is implemented using database validation.

11. AUDIO PLAYER

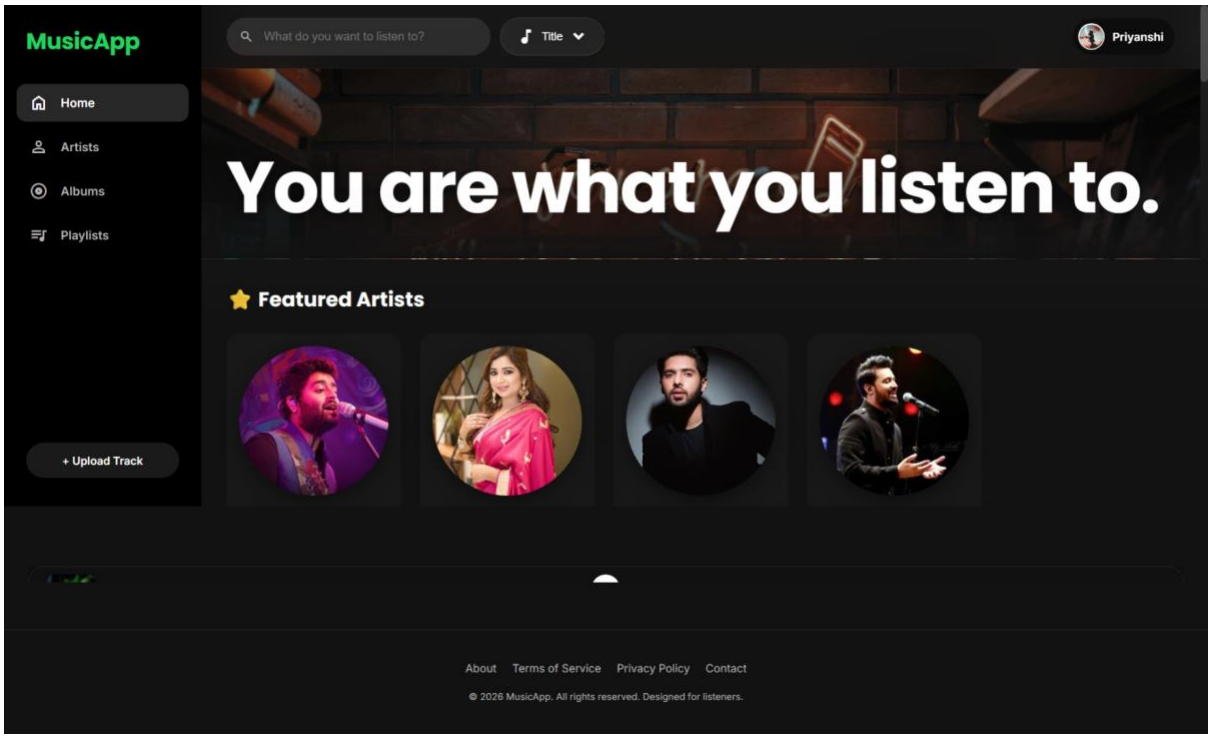
The application includes:

- Play/Pause functionality
- Next/Previous controls
- Progress bar

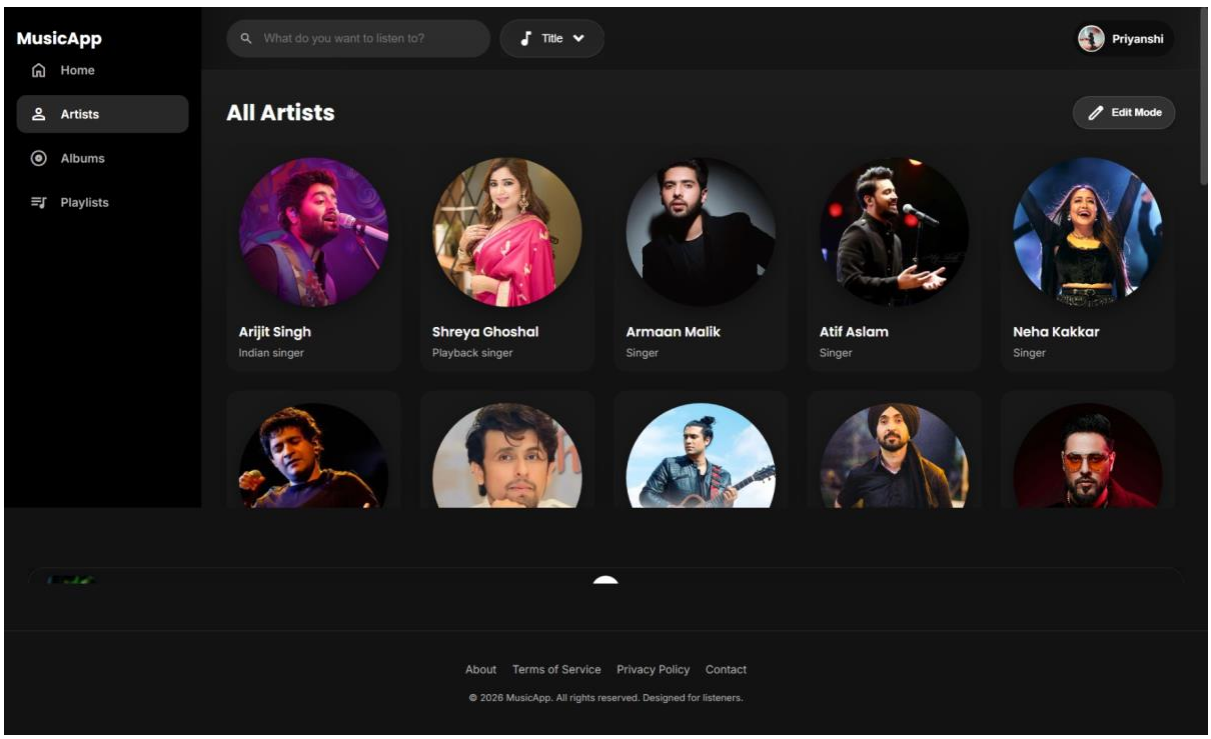
- Playback speed control

12. PROJECT Prototype

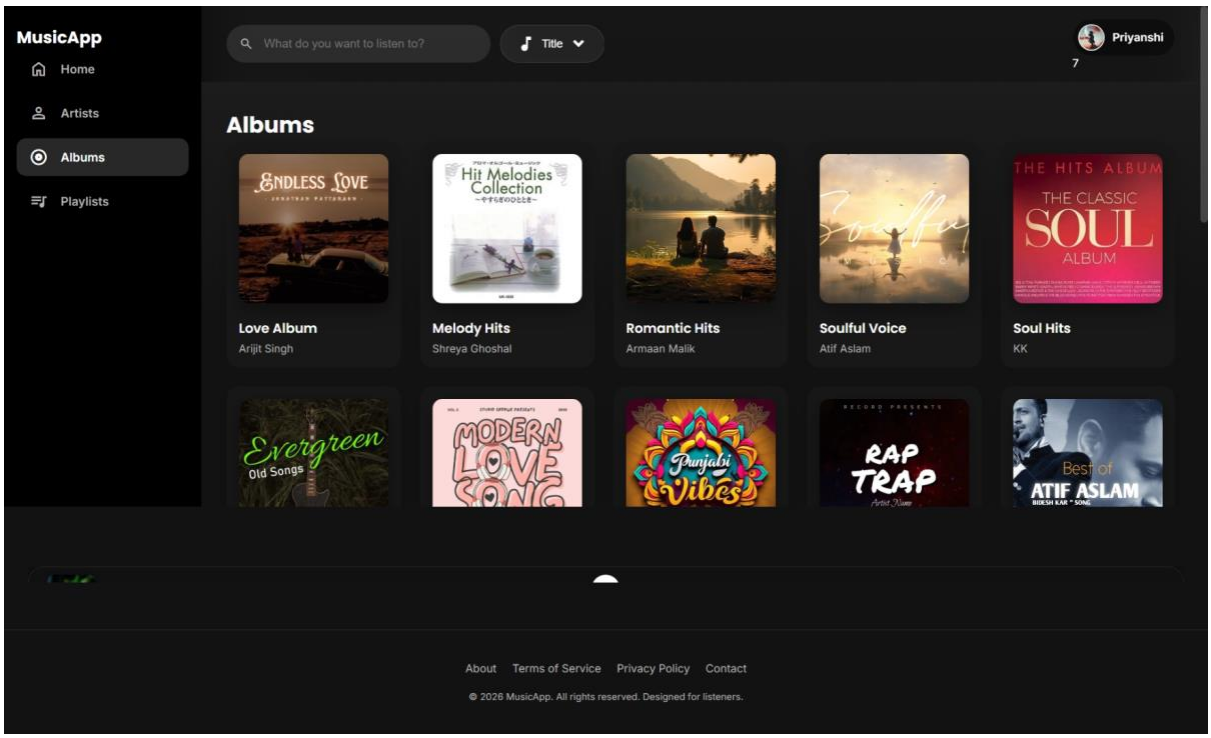
Home



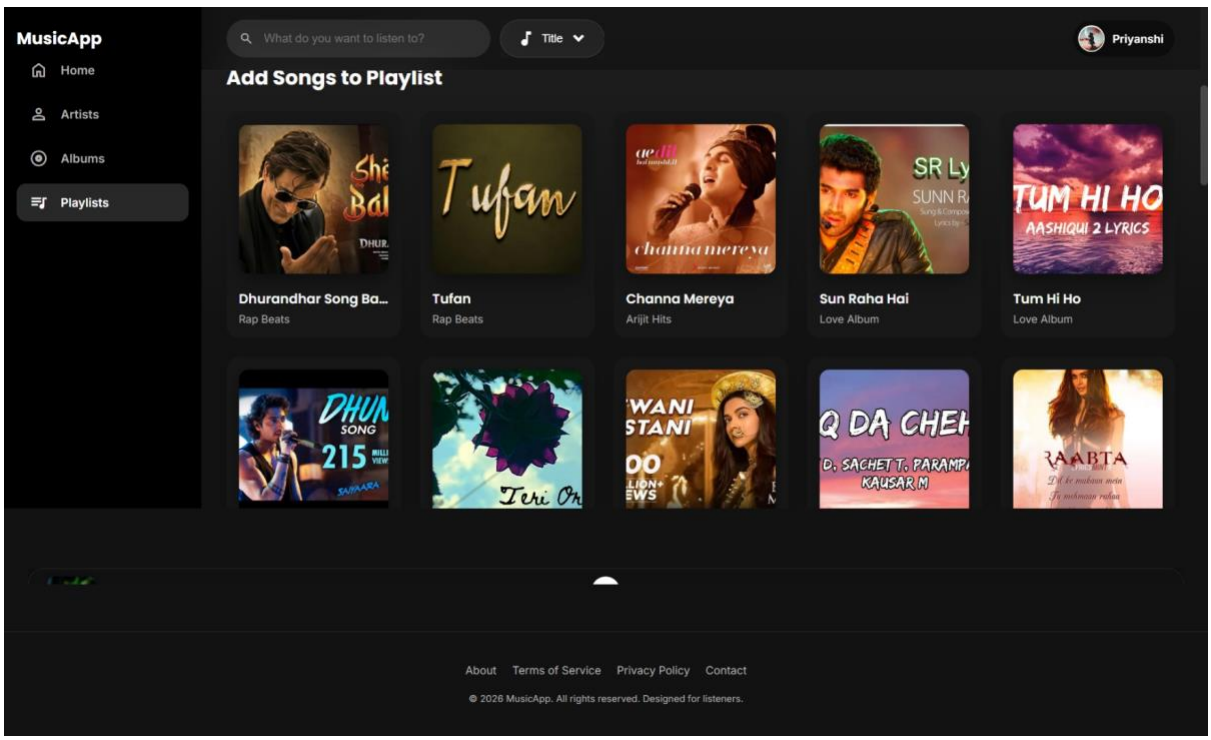
Artists



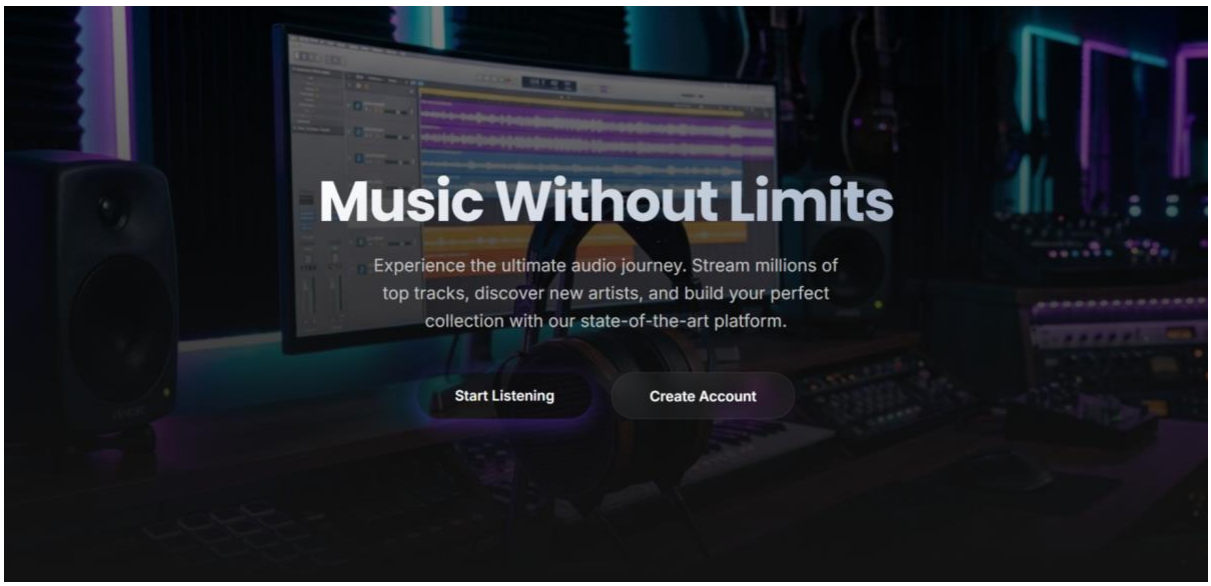
Albums



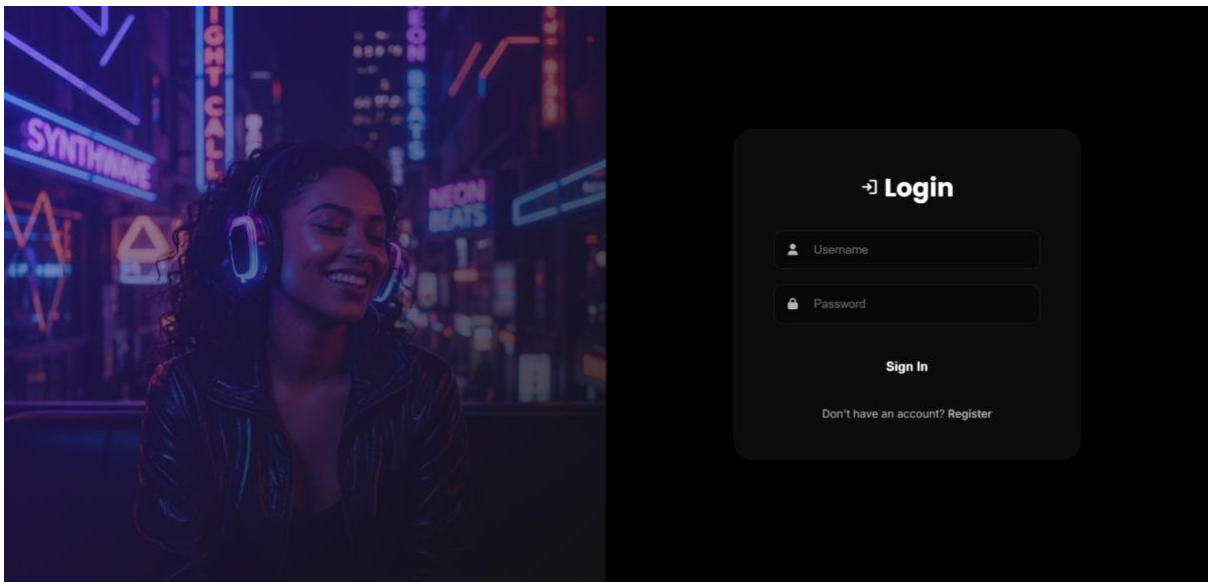
Playlist



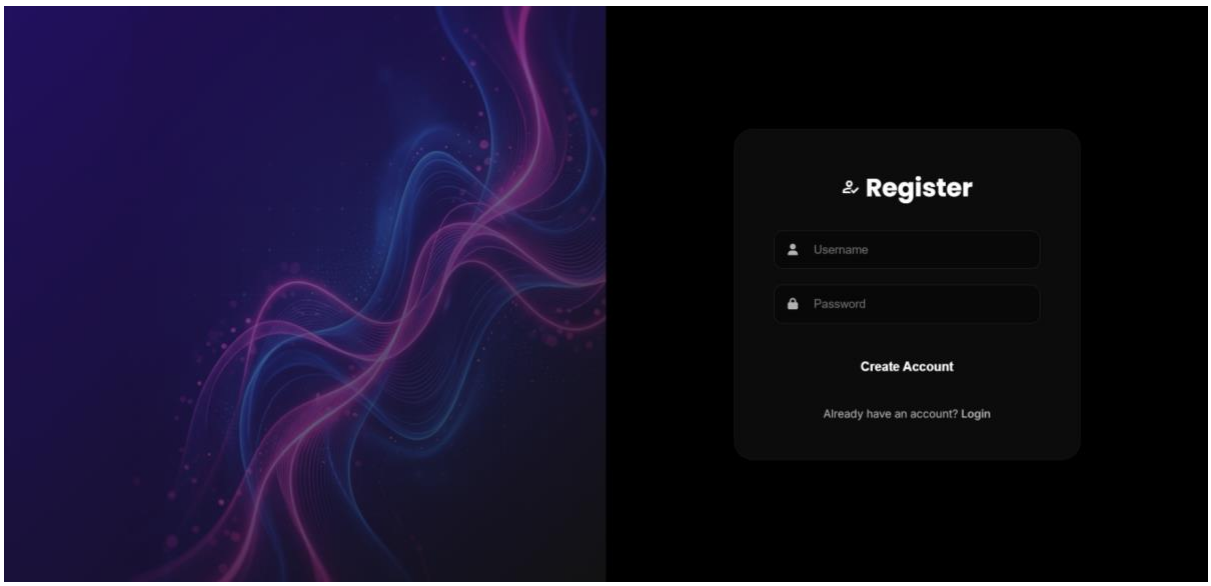
Landing Page



Login



Register



13. CONCLUSION

The Music Library Web Application demonstrates a complete full-stack implementation using modern web technologies. It provides a scalable structure for managing music data and offers a user-friendly interface for interaction.

14. REFERENCES

- Node.js Documentation
- MySQL Documentation
- Express.js Guide
- MDN Web Docs